

How to Use This SQL Guide

SQL is central to the BI3 technical round - the 'second highest salary' query was explicitly reported.

Each problem provides the schema with sample data, the question, a verified solution query, expected output and an explanation.

Every query has been executed in SQLite and verified.

SQL Problems

Problem 1: Second Highest Salary

Difficulty: Easy | Pattern: Subquery

Schema & Sample Data

```
CREATE TABLE employee (id INTEGER, name TEXT, salary INTEGER);
INSERT INTO employee VALUES (1, 'Kiran', 30000), (2, 'Divya', 55000), (3, 'Arun', 55000), (4, 'Bala', 42000);
```

Question

Find the second highest distinct salary from the employee table.

Solution Query

```
SELECT MAX(salary) AS second_highest
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

Expected Output

```
second_highest
42000
```

Explanation

The subquery returns the maximum salary (55000). The outer query returns the largest salary strictly below it, correctly skipping duplicate top salaries, giving 42000.

Problem 2: Nth Highest Salary using LIMIT/OFFSET

Difficulty: Easy-Medium | Pattern: DISTINCT + ORDER BY + LIMIT

Schema & Sample Data

```
CREATE TABLE emp (id INTEGER, salary INTEGER);
INSERT INTO emp VALUES (1, 100), (2, 200), (3, 200), (4, 300), (5, 400);
```

Question

Find the 3rd highest distinct salary using ORDER BY with LIMIT and OFFSET.

Solution Query

```
SELECT DISTINCT salary
FROM emp
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Expected Output

```
salary
200
```

Explanation

Distinct salaries sorted descending are 400, 300, 200, 100. OFFSET 2 skips the top two (400, 300) and LIMIT 1 returns the next one, 200 - the 3rd highest.

Problem 3: Total Sales per Region

Difficulty: Easy | Pattern: GROUP BY / SUM

Schema & Sample Data

```
CREATE TABLE sales (region TEXT, amount INTEGER);
INSERT INTO sales VALUES ('North',100),('South',200),('North',150),('East',50),('South',100);
```

Question

Compute the total sales amount for each region, ordered by total descending.

Solution Query

```
SELECT region, SUM(amount) AS total_sales
FROM sales
GROUP BY region
ORDER BY total_sales DESC;
```

Expected Output

```
region|total_sales
South|300
North|250
East|50
```

Explanation

GROUP BY region aggregates rows per region and SUM adds their amounts. South = 200 + 100 = 300, North = 100 + 150 = 250, East = 50.

Problem 4: Join Orders with Customers

Difficulty: Easy-Medium | Pattern: INNER JOIN

Schema & Sample Data

```
CREATE TABLE customers (id INTEGER, name TEXT);
CREATE TABLE orders (oid INTEGER, cid INTEGER, amount INTEGER);
INSERT INTO customers VALUES (1,'Ram'),(2,'Sita'),(3,'Lax');
INSERT INTO orders VALUES (10,1,500),(11,1,300),(12,2,700);
```

Question

List each customer name with the total amount they have ordered. Include only customers who have placed orders. Order by name.

Solution Query

```
SELECT c.name, SUM(o.amount) AS total
FROM customers c
JOIN orders o ON c.id = o.cid
GROUP BY c.name
ORDER BY c.name;
```

Expected Output

```
name|total
Ram|800
Sita|700
```

Explanation

An INNER JOIN keeps only customers with matching orders (Lax has none, so is excluded). Grouping by name and summing gives Ram = 500 + 300 = 800 and Sita = 700.

Problem 5: Find Departments With More Than 2 Employees

Difficulty: Medium | Pattern: GROUP BY + HAVING

Schema & Sample Data

```
CREATE TABLE emp (id INTEGER, dept TEXT);
INSERT INTO emp VALUES (1, 'A'), (2, 'A'), (3, 'A'), (4, 'B'), (5, 'B'), (6, 'C');
```

Question

List departments that have more than 2 employees, with their counts. Order by department.

Solution Query

```
SELECT dept, COUNT(*) AS headcount
FROM emp
GROUP BY dept
HAVING COUNT(*) > 2
ORDER BY dept;
```

Expected Output

```
dept|headcount
A|3
```

Explanation

HAVING filters aggregated groups. Department A has 3 employees (> 2) and qualifies; B has 2 and C has 1, so both are excluded.

Problem 6: Monthly Order Totals

Difficulty: Medium | Pattern: GROUP BY on Derived Column

Schema & Sample Data

```
CREATE TABLE orders (id INTEGER, order_date TEXT, amount INTEGER);
INSERT INTO orders VALUES (1, '2024-01-05', 100), (2, '2024-01-20', 200), (3, '2024-02-10', 150), (4, '2024-02-20', 150);
```

Question

Compute the total order amount per month (YYYY-MM), ordered by month.

Solution Query

```
SELECT substr(order_date, 1, 7) AS month,
       SUM(amount) AS total
FROM orders
GROUP BY month
ORDER BY month;
```

Expected Output

```
month|total
2024-01|300
```

2024-02 | 300
2024-03 | 300

Explanation

substr extracts the year-month prefix from the date string, which becomes the grouping key. Each month's amounts are summed: Jan 100+200, Feb 150+150, Mar 300.

BI3 Technologies

SQL Problems

Applied Role(s):

Data Engineer - Trainee

Online Assessment Preparation Material

Version 1.0 • Last Updated: July 2026

Compiled from verified previous-year patterns, community-reported questions and research-backed company-style practice.

Table of Contents

How to Use This SQL Guide	1
SQL Problems	1
Problem 1: Second Highest Salary	1
Problem 2: Nth Highest Salary using LIMIT/OFFSET	1
Problem 3: Total Sales per Region	2
Problem 4: Join Orders with Customers	2
Problem 5: Find Departments With More Than 2 Employees	3
Problem 6: Monthly Order Totals	3